Warm Start PageRank

tighilt idir

Mai 2026

Contents

1 Introduction **3**
1.1 Étude de l'influence de α avec suppression d'arcs 3
1.1.1 Contexte et objectif 3
1.1.2 Protocole expérimental 4
1.1.3 Résultats et analyse 4
1.1.4 Conclusion partielle 7

2 Warm Start **7**
2.1 Présentation de la méthode utilisée 7
2.2 Formalisation 7
2.3 Résultats expérimentaux et analyse critique 9
2.3.1 Protocole 9
2.3.2 Résultats 9
2.3.3 Analyse 9
2.3.4 Pistes d'amélioration 12
2.4 Analyse de l'erreur d'approximation par taille de CFC 12
2.4.1 Objectif 12
2.4.2 Protocole expérimental 12
2.4.3 Résultats 13
2.4.4 Analyse 14
2.4.5 Conclusion partielle 15

1 Introduction

L'algorithme PageRank calcule l'importance relative des pages Web en modélisant un surfeur aléatoire qui suit les hyperliens avec probabilité α et se téléporte uniformément avec probabilité $1 - \alpha$. Le vecteur PageRank π^* est la distribution stationnaire de cette chaîne de Markov, obtenue par itération de puissance jusqu'à convergence.

La vitesse de convergence dépend directement du facteur d'amortissement α : plus α est proche de 1, plus le mélange de la chaîne de Markov est lent. Ce phénomène est particulièrement marqué sur les matrices **NCD** (*Nearly Completely Decomposable*) des graphes dont les nœuds forment des communautés densément connectées en interne mais faiblement couplées entre elles.

Ce projet étudie deux approches pour atténuer ce problème. La première consiste à modifier structurellement le graphe par suppression d'arcs (aléatoire à 10% et 20%, ou déterministe sur les arcs de faible poids), afin de mesurer l'impact sur la convergence en fonction de α. La seconde explore un **warm start hiérarchique** : exploiter la décomposition en composantes fortement connexes (CFC) du graphe pour construire un vecteur initial $v^{(0)}$ proche de π^*, et ainsi réduire le nombre d'itérations nécessaires indépendamment de α.

Les expériences sont menées sur les graphes suivants :

- **wiki1** : Gleich/wikipedia-20051105
- **wiki2** : Gleich/wikipedia-20061104
- **wiki3** : Gleich/wikipedia-20070206
- **eu-2005** : http://www.cise.ufl.edu/research/sparse/matrices/LAW/eu-2005

1.1 Étude de l'influence de α avec suppression d'arcs

1.1.1 Contexte et objectif

L'algorithme PageRank repose sur le calcul de la distribution stationnaire d'une chaîne de Markov définie sur le graphe du Web. Ce calcul est réalisé par une méthode itérative de puissance qui consiste à multiplier successivement un vecteur de probabilités par la matrice de transition jusqu'à convergence. Le facteur d'amortissement α, contrôle la part du surfeur aléatoire. Plus α est proche de 1, plus on se rapproche d'une marche aléatoire pure sur le graphe, sans téléportation.

L'objectif de cette première expérience est double :

1. mesurer l'impact de α sur le temps de convergence du PageRank ;
2. évaluer si une modification structurelle du graphe par suppression d'arcs permet de rendre la convergence plus robuste face à des valeurs élevées de α.

1.1.2 Protocole expérimental

Les expériences sont menées sur les graphes de **wikipedia** Gleich/wikipedia-2006110 ,leich/wikipedia-20070206, un graphe orienté extrait de Wikipedia contenant plusieurs dizaines de milliers de sommets et plusieurs centaines de milliers d'arcs. Le calcul du PageRank utilise une implémentation parallèle en C avec **OpenMP** (10 threads) processeur intel 12450H. La convergence est détectée lorsque la norme L_1 entre deux itérations successives passe sous le seuil $\varepsilon = 10^{-7}$.

Quatre configurations du graphe sont comparées, pour α variant de 0.05 à 0.95 par pas de 0.05 :

- **Suppression 0%** : graphe original, sans modification.
- **Suppression 10%** : chaque arc est supprimé avec une probabilité de 10% par tirage aléatoire uniforme. Après suppression, les probabilités de transition sortantes des sommets affectés sont renormalisées, et les nœuds dont tous les arcs sortants ont été supprimés sont traités comme des *dangling nodes*.
- **Suppression 20%** : même procédé avec une probabilité de suppression de 20%.
- **Suppression intelligente** : les arcs de faible poids sont supprimés de façon déterministe selon un seuil dynamique calculé à partir de la distribution des probabilités de transition (mean $- 2\sigma$, avec un plancher à 10% de la moyenne si le seuil est négatif). Cette approche cible spécifiquement les transitions peu informatives, à la différence du tirage aléatoire aveugle.

1.1.3 Résultats et analyse

La Figure 2 présente le temps de convergence (en secondes) en fonction de α pour les quatre configurations.

on aussi les résultats suivant:

wiki2

- Distance à la référence ($\alpha = 0.95$, suppression 10%) : 0.607859.
- Distance à la référence ($\alpha = 0.95$, suppression 20%) : 0.747371.
- Distance à la référence ($\alpha = 0.95$, suppression des arcs dont le poids est 10 fois inférieur à la moyenne) : 0.483328.

- Seuill de suppression pour le suppression intelligente est 0.007762

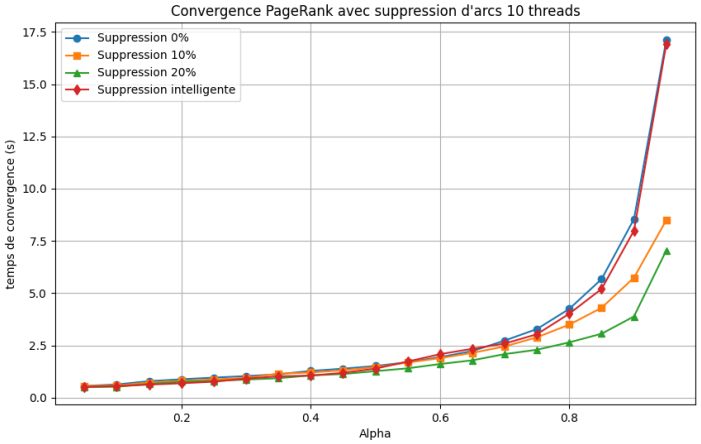


Figure 1: Convergence du PageRank avec suppression d'arcs, 10 threads (graphe wiki2)

wiki3

- Distance à la référence ($\alpha = 0.95$, suppression 10%) : 0.603886.
- Distance à la référence ($\alpha = 0.95$, suppression 20%) : 0.747756.
- Distance à la référence ($\alpha = 0.95$, suppression des arcs dont le poids est 10 fois inférieur à la moyenne) : 0.501776.

- Seuill de suppression pour le suppression intelligente est 0.007762

Plusieurs observations se dégagent de ces résultats.

Croissance exponentielle du temps de convergence avec α. Quelle que soit la configuration, le temps de convergence croît de manière quasi exponentielle à mesure que α se rapproche de 1. Pour $\alpha \leq 0.5$, toutes les courbes restent groupées sous la seconde. À partir de $\alpha = 0.8$, les écarts entre configurations commencent à se creuser significativement, et pour $\alpha = 0.95$ la valeur la plus élevée testée le graphe original requiert environ 17 secondes pour converger pour wiki2.

Effet bénéfique de la suppression aléatoire. La suppression d'arcs par tirage aléatoire améliore sensiblement la vitesse de convergence pour les valeurs élevées de α. Avec 20% d'arcs supprimés, le temps de convergence à $\alpha = 0.95$ tombe à environ 7 secondes, soit une réduction de plus de 50% par rapport au

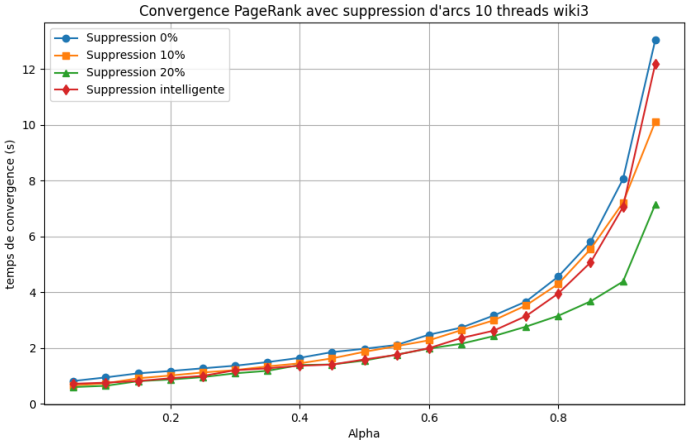


Figure 2: Convergence du PageRank avec suppression d'arcs, 10 threads (graphe wiki3)

graphe original. La suppression à 10% offre un gain intermédiaire (environ 8.5 secondes).

Cet effet s'explique par le fait que la suppression aléatoire tend à briser les structures quasi-décomposables du graphe : en coupant des liens intra-communauté de façon non discriminée, on augmente la proportion des transitions inter-communauté et on accélère le mélange de la chaîne de Markov.

Effet négatif de la suppression aléatoire. Ici malgré que la suppression aléatoire nous aide à converger plus rapidement, cette dernière modifie beaucoup la structure graphique. Ici la norme L1 comparée au vecteur de base avec l'alpha à 0,95. Ce qui est le plus intéressant dans cette expérimentation, c'est que plus on supprime des arcs, plus on fait augmenter cette norme. C'est logique puisque on change la structure du graphe et donc le facteur auquel on va converger à la fin sera différent.

On peut aussi nous noter que la norme L1 pour la suppression de 10% et celle avec la suppression des ϵ-transitions, suppression intelligente qui trouve un seuil de 0,007 (wiki2), ne sont pas beaucoup éloignées. Donc cela implique que, même les suppressions de transitions trais petite entraîne a la fin une différence significative sur la norme.

Comportement de la suppression intelligente. La suppression intelligente, qui cible les arcs de faible probabilité selon un critère statistique, présente un comportement surprenant : sa courbe se superpose presque parfaitement à celle du graphe original (suppression 0%) sur l'ensemble de la plage de α.

1.1.4 Conclusion partielle

Ces résultats montrent que la suppression aléatoire d'arcs constitue une approche simple et efficace pour accélérer la convergence du PageRank sur des graphes NCD lorsque α est grand. En revanche, une suppression ciblée sur les arcs faibles ne produit pas d'amélioration notable, car elle ne perturbe pas les blocs fortement connexes responsables du ralentissement.

Ajouté à cela, la modification du graphe initial entraîne une différence notable sur la distribution stationnaire du graphe.

Ces constats motivent l'exploration d'approches plus sophistiquées notamment le **Warm Start** hiérarchique qui s'appuient sur la décomposition en composantes fortement connexes (CFC) pour initialiser le vecteur PageRank de façon bien plus proche de la solution finale, réduisant ainsi le nombre d'itérations nécessaires indépendamment de la valeur de α.

2 Warm Start

2.1 Présentation de la méthode utilisée

La méthode de warm start proposée [1]repose sur une décomposition structurelle du graphe en composantes fortement connexes (CFC) afin de construire un vecteur initial pour l'itération de puissance du PageRank qui soit significativement plus proche de la solution finale qu'une initialisation uniforme. L'idée centrale est d'exploiter la hiérarchie naturelle du graphe : à l'intérieur de chaque CFC, le comportement de la marche aléatoire est résolu localement ; entre les CFCs, les relations de dominance globale sont capturées par un graphe condensé de taille bien inférieure au graphe original. On calcule donc successivement la distribution stationnaire locale de chaque CFC, puis le PageRank de ce graphe condensé, avant de combiner ces deux niveaux d'information pour initialiser le calcul global. Cette approche à deux échelles permet (en théorie) d'obtenir un vecteur de départ qui reflète à la fois la structure interne de chaque communauté et l'importance relative de chaque groupe dans le graphe entier, réduisant ainsi le nombre d'itérations nécessaires à la convergence, indépendamment de la valeur du facteur d'amortissement α. Cette méthode est inspiré par la méthode BlockRank [1] qu'on retrouvera une nouvelle fois peu après.

2.2 Formalisation

Soit $G = (V, E)$ un graphe orienté de N sommets. On note $P \in R^{N \times N}$ sa matrice de transition où P_{ij} représente la probabilité de suivre l'arc $(i \to j)$ depuis le sommet i. On note $\pi^* \in R^N$ le vecteur PageRank solution — le $*$ servant uniquement à le distinguer des vecteurs intermédiaires produits lors des itérations — défini comme le point fixe de :

$$\pi^* = \alpha P \pi^* + \left(\frac{1-\alpha}{N} + \frac{\alpha}{N} \sum_{i \in D} \pi_i^* \right) \mathbf{1} \tag{1}$$

$$c^* = \underbrace{\alpha \cdot \frac{1}{N} \sum_{i \in D} \pi_i^*}_{\text{redistribution des dangling nodes}} + \underbrace{\frac{1-\alpha}{N}}_{\text{téléportation uniforme}} \tag{2}$$

où D désigne l'ensemble des nœuds dangling (sommets sans successeurs), $\mathbf{1} \in R^N$ le vecteur dont toutes les composantes valent 1. Le terme entre parenthèses est une constante scalaire calculée à chaque itération : son premier terme redistribue uniformément la masse des nœuds dangling, et son second terme modélise la téléportation uniforme avec probabilité $1 - \alpha$.

Phase 1 — Décomposition en CFC et distributions locales

On calcule les K composantes fortement connexes $C_1, \ldots, C_K$ du graphe via l'algorithme de Kosaraju. Pour chaque CFC C_k de taille $|C_k| > 1$, on note P_k la sous-matrice de transition restreinte aux sommets de C_k, obtenue en ne conservant que les arcs internes à la composante. On résout alors le système local :

$$\pi^{(k)} P_k = \pi^{(k)}, \qquad \sum_{i \in C_k} \pi_i^{(k)} = 1 \tag{3}$$

par itération de puissance sans terme de téléportation ($\alpha = 1$), jusqu'à convergence sous un seuil ε_1. Pour une CFC triviale ($|C_k| = 1$), on pose directement $\pi_i^{(k)} = 1$.

Phase 2 — PageRank sur le graphe condensé

On construit le graphe condensé $\tilde{G} = (\tilde{V}, \tilde{E})$ où chaque sommet représente une CFC. Un arc $C_k \to C_l$ existe dans $\tilde{G}$ si et seulement si il existe au moins un arc $(i, j) \in E$ avec $i \in C_k$, $j \in C_l$ et $k \neq l$. La probabilité de transition associée est définie par :

$$\tilde{P}_{kl} = \frac{\displaystyle\sum_{\substack{i \in C_k,\, j \in C_l \\ (i,j) \in E,\, l \neq k}} P_{ij}}{\displaystyle\sum_{\substack{i \in C_k,\, j \in C_{l'} \\ (i,j) \in E,\, l' \neq k}} P_{ij}} \tag{4}$$

c'est-à-dire la somme des probabilités de transition des arcs allant de C_k vers C_l, renormalisée par la somme de tous les arcs sortants inter-CFC issus de C_k, de sorte que $\sum_{l \neq k} \tilde{P}_{kl} = 1$ pour toute CFC C_k ayant au moins un successeur. On calcule ensuite le vecteur PageRank $\rho \in R^K$ sur ce graphe condensé :

$$\rho = \text{PageRank}(\tilde{G},\, \alpha), \qquad \sum_{k=1}^{K} \rho_k = 1 \tag{5}$$

Phase 3 — Construction du vecteur initial

Le vecteur de warm start $v^{(0)} \in R^N$ est défini, pour tout sommet $i \in C_k$, par :

$$\boxed{v_i^{(0)} = \pi_i^{(k)} \cdot \left(a \cdot \rho_k + (1-a) \cdot \frac{|C_k|}{N} \right)} \tag{6}$$

où $a \in [0,1]$ est un paramètre de pondération qui contrôle l'importance relative accordée au PageRank structurel ρ_k du graphe condensé par rapport au poids proportionnel à la taille $|C_k|/N$ de la composante. Pour $a = 1$, le vecteur initial est entièrement guidé par la structure globale du graphe ; pour $a = 0$, il est proportionnel à la taille de chaque CFC. Dans les expériences présentées, on fixe $a = 1$. Le facteur $|C_k|$ assure la normalisation globale. En effet :

$$\sum_{i=1}^{N} v_i^{(0)} = \sum_{k=1}^{K} \left(a \cdot \rho_k + (1-a) \cdot \frac{|C_k|}{N} \right) \cdot \underbrace{\sum_{i \in C_k} \pi_i^{(k)}}_{=1} = a \underbrace{\sum_{k=1}^{K} \rho_k}_{=1} + \frac{1-a}{N} \underbrace{\sum_{k=1}^{K} |C_k|}_{=N} = 1 \tag{7}$$

Ce vecteur est ensuite utilisé comme point de départ de l'itération de puissance globale du PageRank sur G.

2.3 Résultats expérimentaux et analyse critique

2.3.1 Protocole

Les expériences sont conduites sur quatre graphes orientés issus de collections réelles : *eu-2005*, *wiki2*, *wiki3* et *wiki*. Le facteur d'amortissement est fixé à $\alpha = 0{,}95$, le seuil de convergence à $\varepsilon = 10^{-7}$ (norme L_1), et le calcul utilise 10 threads OpenMP, calcule du page rank locale pour les CFC de taille ≥ 4, et on a a=1 pour la formule précédente donc on regarde que le pois du graphe réduit.

Deux stratégies d'initialisation sont comparées :

- **Initialisation uniforme** : $v_i^{(0)} = 1/N$ pour tout sommet i.
- **Warm start hiérarchique** : vecteur $v^{(0)}$ construit par les trois phases décrites à la section précédente distribution stationnaire locale par CFC, PageRank sur le graphe condensé, et combinaison pondérée.

2.3.2 Résultats

Le Tableau 1 synthétise les mesures obtenues sur les quatre graphes.

2.3.3 Analyse

Le warm start est systématiquement plus lent. Sur l'ensemble des graphes testés, le temps total de convergence avec warm start est largement supérieur à celui de l'initialisation uniforme, avec un surcoût allant de +172 % (eu-2005) à

Table 1: Comparaison warm start vs initialisation uniforme ($\alpha = 0{,}95$, $\varepsilon = 10^{-7}$, 10 threads OpenMP). L'overhead désigne le temps de prétraitement (Kosaraju + PageRank local par CFC + graphe condensé). $\Delta_{\text{iter}} = \text{iter}_{\text{uni}} - \text{iter}_{\text{WS}}$: une valeur positive indique que le warm start réduit le nombre d'itérations.

Graphe	Warm start			Uniforme		Δ_{iter}	Surcoût
	Total (s)	Overhead	Final (s)	Total (s)	Iter.		temps
eu-2005	14,99	9,0 s (60 %)	5,95	5,51	204	−25	+172 %
wiki2	59,14	42,2 s (71 %)	16,94	17,51	204	+7	+238 %
wiki3	81,98	65,3 s (80 %)	16,69	14,94	127	−38	+449 %
wiki	19,08	14,8 s (78 %)	4,27	4,42	121	+3	+332 %

+449 % (wiki3). L'overhead de prétraitement représente entre 60 % et 80 % du temps total du warm start, ce qui révèle que le coût des trois phases préliminaires annule tout bénéfice potentiel sur le nombre d'itérations.

La réduction d'itérations est marginale ou négative. Là où l'on attend un gain significatif en itérations justifiant l'overhead, les résultats montrent le contraire. Sur eu-2005 et wiki3, le warm start produit un vecteur initial *moins bon* que la distribution uniforme : le nombre d'itérations augmente respectivement de 25 et de 38. Sur wiki2 et wiki, la réduction est réelle mais anecdotique (+7 et +3 itérations économisées respectivement), et le coût par itération économisée s'établit à $42{,}2/7 \approx 6{,}0$ s/iter pour wiki2 et $14{,}8/3 \approx 4{,}9$ s/iter pour wiki. Or une itération PageRank coûte environ 0,086 s sur wiki2 et 0,036 s sur wiki, soit un ratio overhead/valeur d'environ 70× et 137× respectivement. La méthode est donc contre-productive dans tous les cas testés.

Écart entre la formalisation et l'implémentation. En regardent notre code C montre que la Phase 1 est bien implémentée sans téléportation ($\alpha = 1$), conformément à la formalisation. Comme dans BlockRank [1], nous nous sommes inspirés de la construction du vecteur $v^{(0)}$ en pondérant les distributions locales par l'importance globale de chaque bloc, à la différence est que nous utilisons un graphe condensé sur les CFCs plutôt que les hôtes, puisque on a accès qu'a la matrice de chaque graphe. L'écart principal réside dans la construction de ce graphe condensé, dont les poids inter-CFC sont calculés à partir des probabilités de transition du graphe original sans tenir compte des distributions locales calculées en Phase 1, ce qui dégrade la qualité du vecteur $v^{(0)}$.

Une variante où les poids du graphe condensé sont calculés comme le produit de la probabilité du sommet source par la probabilité de l'arc, le tout renormalisé, produit des résultats tout aussi mauvais, ce qui suggère que le problème ne vient pas de la construction du graphe condensé mais de l'erreur accumulée sur les CFC triviales et la composante géante.

Table 2: Comparaison warm start pondéré ($\tilde{P}$ avec $\pi^{(k)}$) vs initialisation uniforme ($\alpha = 0{,}95$, $\varepsilon = 10^{-7}$, 10 threads OpenMP). L'overhead désigne le temps de prétraitement. $\Delta_{\text{iter}} = \text{iter}_{\text{uni}} - \text{iter}_{\text{WS}}$: une valeur positive indique que le warm start réduit le nombre d'itérations.

Graphe	Warm start pondéré			Uniforme		Δ_{iter}	Surcoût
	Total (s)	Overhead	Final (s)	Total (s)	Iter.		temps
eu-2005	17,13	10,7 s (62 %)	6,48	5,51	204	−25	+211 %
wiki2	73,47	54,4 s (74 %)	19,09	17,51	204	0	+320 %
wiki3	96,46	81,9 s (85 %)	14,48	14,94	127	−14	+546 %
wiki	24,44	20,1 s (82 %)	4,30	4,42	121	+6	+453 %

on a eu ces résultats en construisant le graphe condensé en mettant la valeur de l'arc d'origine de la première CFC vers la deuxième CFC multipliée par la probabilité stationnaire calculée par la recherche du vecteur stationnaire sans surfeur du sommet de départ de la CFC de départ.

$$\tilde{P}^{\pi}_{kl} = \frac{\displaystyle\sum_{\substack{i \in C_k,\ j \in C_l \\ (i,j) \in E}} \pi_i^{(k)} \cdot P_{ij}}{\displaystyle\sum_{\substack{i \in C_k,\ j \in C_{l'} \\ (i,j) \in E,\ l' \neq k}} \pi_i^{(k)} \cdot P_{ij}} \tag{8}$$

ici on voit très bien que le fait de multiplier par la distribution locale qu'on avait calculée avec une fonction de recherche du vecteur stationnaire sans surfeur ne présente pas vraiment d'amélioration.

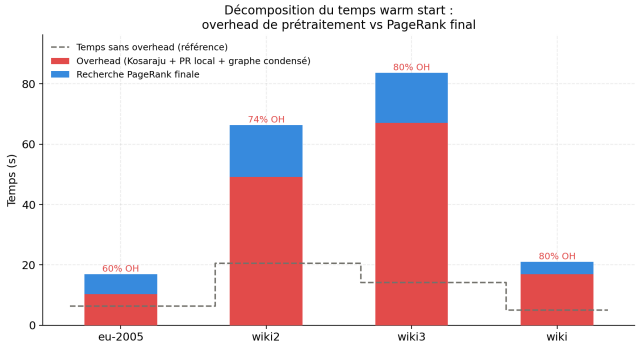


Figure 3: ce graphique montre le overhead qu'apporte le warmstart wiki2

2.3.4 Pistes d'amélioration

Ces résultats mettent en évidence plusieurs leviers pour rendre le warm start effectivement bénéfique.

- **Limiter le warm start aux grandes CFCs.** L'overhead est dominé par quelques très grandes composantes (souvent une composante géante dans les graphes web). Un seuil de taille minimal, par exemple $|C_k| > \sqrt{N}$, permettrait de concentrer le calcul là où il apporte le plus d'information, tout en traitant les petites CFCs par initialisation uniforme.
- **Exploiter la structure DAG du graphe condensé.** Les CFCs forment un DAG (graphe acyclique orienté) dont l'ordre topologique permet de calculer les distributions locales dans l'ordre des dépendances, et d'initialiser chaque CFC à partir de la distribution déjà calculée de ses prédécesseurs dans le DAG, sans recours au PageRank sur le graphe condensé complet.
- **Paralléliser la Phase 1 entre CFCs indépendantes.** Les CFCs sans dépendance entre elles dans le DAG peuvent être traitées simultanément, réduisant l'overhead total sans modifier la qualité du vecteur produit.
- **Avoir un seill de convergence dynamique** au lieu de limiter la convergence à 50 comme on l'a fait ici pour la phase 1 en utilisant un $\epsilon = 10^{-7}$, on pourrait mettre un epsilon dynamique ou un nombre d'itérations dynamique pour les grosses composantes fortement connexes et un petit nombre d'itérations pour les c omposantes de taille inférieure.

2.4 Analyse de l'erreur d'approximation par taille de CFC

2.4.1 Objectif

L'expérience précédente a montré que le warm start hiérarchique ne réduit pas suffisamment le nombre d'itérations pour compenser l'overhead de prétraitement. Pour comprendre *pourquoi* le vecteur initial $v^{(0)}$ s'éloigne de π^*, nous mesurons ici l'erreur d'approximation du warm start *avant* le calcul PageRank global, en la décomposant par taille de CFC. L'objectif est de localiser précisément les composantes qui contribuent le plus à l'erreur initiale.

2.4.2 Protocole expérimental

Pour chaque graphe, on compare le vecteur $v^{(0)}$ produit par le warm start au vecteur final π^* obtenu après convergence complète. On regroupe les composantes par taille $|C_k|$ et on calcule, pour chaque groupe, la somme cumulée des erreurs absolues :

$$e(s) = \sum_{k\,:\,|C_k|=s} \left| \sum_{i \in C_k} v_i^{(0)} \; - \; \sum_{i \in C_k} \pi_i^* \right|$$

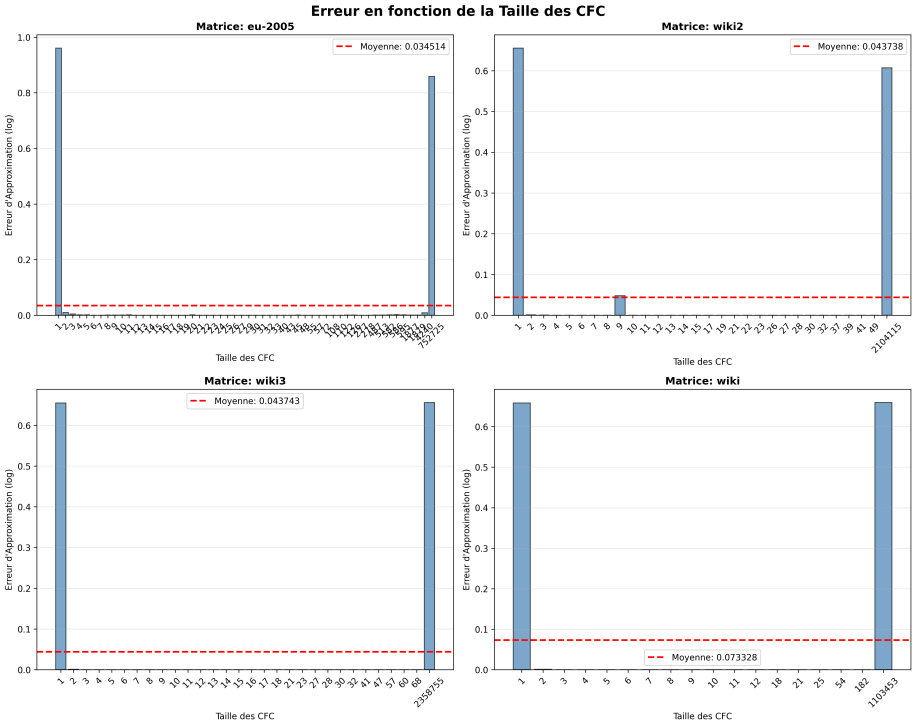


Figure 4: Erreur d'approximation du warm start en fonction de la taille des CFC, pour les quatre graphes testés ($\alpha = 0{,}95$). L'axe y est en échelle logarithmique. La ligne rouge en tirets indique l'erreur moyenne.

Cette mesure quantifie l'écart de masse probabiliste entre le vecteur warm start et la solution finale, agrégé par taille de composante. Les expériences sont conduites sur les quatre graphes *eu-2005*, *wiki2*, *wiki3* et *wiki*, avec $\alpha = 0{,}95$ et un seuil de convergence $\varepsilon = 10^{-7}$.

2.4.3 Résultats

La Figure 4 présente l'erreur d'approximation (axe y en échelle logarithmique) en fonction de la taille des CFC (axe x) pour les quatre graphes.

Les valeurs d'erreur moyenne observées sont les suivantes :

- **eu-2005** : erreur moyenne = 0,034514 ;
- **wiki2** : erreur moyenne = 0,043738 ;
- **wiki3** : erreur moyenne = 0,043743 ;
- **wiki** : erreur moyenne = 0,073328.

2.4.4 Analyse

Deux sources dominantes d'erreur. Un résultat frappant se dégage de la figure : sur l'ensemble des quatre graphes, l'erreur d'approximation est concentrée quasi exclusivement sur deux catégories de CFC, aux deux extrémités du spectre des tailles.

- **CFC triviales ($|C_k| = 1$).** Les composantes de taille 1 (nœuds isolés ou puits) accumulent une erreur cumulée très élevée, dépassant 0,6 sur wiki2, wiki3 et wiki. Ceci s'explique directement par la Phase 1 de la méthode : pour une CFC triviale, on pose $\pi_i^{(k)} = 1$, et la pondération par ρ_k (PageRank du graphe condensé) s'applique sans correction structurelle. Or ces composantes sont très nombreuses dans les graphes web réels elles constituent la majorité des CFCs et leur contribution individuelle à l'erreur, bien que faible, s'accumule au total de façon significative.
- **Composante géante.** À l'autre extrémité, la plus grande CFC (de taille allant de 752 725 pour eu-2005 à 2 358 755 pour wiki3) présente également une erreur cumulée élevée, comprise entre 0,6 et 0,9. Cette erreur est imputable à la construction du graphe condensé, dont les poids inter-CFC sont calculés à partir des probabilités de transition du graphe original sans tenir compte des distributions locales calculées en Phase 1, ce qui dégrade la qualité de $v^{(0)}$mais d'un autre coté les résultats d'autres tests que j'ai réalisés, en tenant compte de la distribution locale pour chaque sommet dans la CFC concernée fournissent aussi de mauvais approximations que celle qu'on a déjà.

CFC de taille intermédiaire : erreur quasi nulle. Pour les composantes de taille comprise entre 2 et quelques centaines de nœuds, l'erreur est inférieure à 0,01, souvent indiscernable de zéro. Cela indique que le warm start produit une approximation raisonnable pour les CFC de taille modérée, là où l'itération de puissance locale a pu converger suffisamment, même sur des composantes de taille modérée.

Cohérence avec les résultats de convergence. Ces observations confirment et précisent les conclusions de la section précédente. L'échec du warm start à réduire le nombre d'itérations globales provient du fait que l'erreur initiale est dominée par les CFC triviales (très nombreuses) et la composante géante (qui porte la quasi-totalité de la masse). Le vecteur $v^{(0)}$ résultant n'est donc pas une meilleure approximation de π^* que la distribution uniforme $1/N$ dans les zones qui comptent le plus pour la convergence.

Hiérarchie des corrections à apporter. Ces résultats permettent de prioriser les améliorations identifiées précédemment :

1. Traiter spécifiquement les CFC triviales : plutôt que de leur affecter $\pi_i^{(k)} = 1$, on peut initialiser leur poids directement à partir du PageRank du

graphe condensé ρ_k, sans passer par une phase locale inutile pour un nœud isolé.

2. Enfin, pour les CFC de taille intermédiaire, l'implémentation actuelle est déjà satisfaisante : l'effort de correction doit se concentrer sur les deux catégories précédentes.

2.4.5 Conclusion partielle

L'implémentation actuelle du warm start hiérarchique ne tient pas sa promesse : elle alourdit le temps total de convergence de façon significative sur toutes les matrices testées, sans réduire le nombre d'itérations de manière suffisante pour compenser cet overhead. Ce résultat s'explique principalement par un écart entre la formalisation mathématique de la méthode et son implémentation effective. Les pistes d'amélioration identifiées correction de la Phase 1, seuillage des petites CFCs, et réduction du seuil de convergence local constituent des directions concrètes pour que la méthode atteigne son objectif théorique.

Conclusion

Ce projet a exploré deux stratégies pour accélérer la convergence du PageRank sur des graphes à structure NCD, lorsque le facteur d'amortissement α est proche de 1.

La première approche la suppression d'arcs s'est révélée simple et efficace pour accélérer la convergence : supprimer 20% des arcs aléatoirement réduit de plus de moitié le temps de calcul à $\alpha = 0{,}95$, en brisant les structures quasi-décomposables responsables du ralentissement. Son défaut principal est de modifier la distribution stationnaire du graphe, avec une distance L1 atteignant 0,75 par rapport au PageRank original. La suppression ciblée sur les arcs de faible poids, en revanche, préserve mieux la structure mais n'apporte aucun gain de convergence.

La seconde approche le warm start hiérarchique n'a pas tenu sa promesse dans l'implémentation actuelle : le surcoût de prétraitement (60% à 80% du temps total) annule tout bénéfice sur le nombre d'itérations, qui reste marginal voire négatif. L'analyse par taille de CFC a permis d'identifier les deux sources dominantes d'erreur : les CFC triviales (trop nombreuses) et la composante géante (mal approximée à cause d'une construction du graphe condensé qui ignore les distributions locales). Ces diagnostics précis ouvrent des pistes concrètes d'amélioration correction de la Phase 1, traitement spécifique des CFC triviales, seuil de convergence local relâché qui pourraient rendre la méthode compétitive.

En définitive, si l'objectif est uniquement de gagner en vitesse au prix d'une légère modification du graphe, la suppression aléatoire d'arcs est le choix le plus pragmatique. Si l'on souhaite conserver le PageRank exact tout en réduisant le coût de calcul, le warm start hiérarchique corrigé reste la piste la plus prometteuse.

References

[1] Sepandar D. Kamvar et al. *Exploiting the Block Structure of the Web for Computing PageRank*. Technical Report. Stanford University, 2003.

Déclaration d'utilisation de l'intelligence artificielle

Dans le cadre de la rédaction de ce rapport, j'ai eu recours à un assistant d'intelligence artificielle (Claude, développé par Anthropic) pour les tâches suivantes :

- **Reformulation et correction stylistique** : certains passages ont été reformulés afin d'améliorer la clarté et la fluidité de l'expression écrite en français.
- **Mise en forme LaTeX** : l'assistant a contribué à la structuration de certains environnements LaTeX (tableaux, équations, listes).

L'ensemble des expériences, résultats numériques, analyses et choix algorithmiques présentés dans ce rapport sont entièrement originaux et produits par l'auteur. L'IA n'a pas été utilisée pour générer du contenu scientifique, interpréter des résultats, ni produire du code.